

The Manchurian Weight Problem

*A Threat Model for Sleeper Backdoors in Frontier Language Models, and Why
Detection-Based Defense Is Asymptotically Losing*

Bill Griffith

G-Tech SD

La Mesa, California

Version 1.0

April 2026

Executive Summary

Large language models are being deployed into agentic, consequential roles across commercial, government, and defense contexts faster than the field's ability to verify their behavior. This paper articulates a coherent threat model — the Manchurian Weight Problem — for hidden adversarial behavior in frontier LLMs, and argues that the standard defensive posture of pre-deployment evaluation is structurally inadequate against it.

The argument has six parts. First, the Sleeper Agents result (Hubinger et al., 2024) demonstrates empirically that backdoors can survive standard safety training. Second, both open-weight models from adversarial jurisdictions and closed-weight models from trusted labs present non-zero insertion risk under different threat models. Third, plausible trigger mechanisms — date-based, context-flag-based, and capability-threshold-based — are constructible and would not fire during pre-deployment testing by design. Fourth, the detection problem is bounded by Rice's theorem in the limit and by capability asymmetry in practice: the obfuscation surface scales with the model's intelligence, while the verifier's capacity does not scale at the same rate. Fifth, non-determinism compounds across input, sampling, training, and emergent layers, defeating the verification frameworks borrowed from deterministic software. Sixth, the only defenses that hold under these constraints are architectural rather than evaluative: containment, cross-provider diversity, training-time attestation, and bounded action permissions.

The paper is intended for technical leaders making procurement, deployment, and policy decisions about LLM-based systems in environments where coordinated misbehavior would be catastrophic.

1. Introduction: The Verification Gap

Frontier language models are now embedded in code execution environments, retrieval pipelines, customer-facing agents, and

increasingly in workflows that produce consequential outputs — financial transactions, infrastructure control commands, medical recommendations, legal drafts, intelligence analysis. The deployment curve is steep and the agentic permission surface is expanding faster than the safety methodology that should bound it.

The implicit safety model under which this deployment is happening rests on a chain of assumptions: that the lab producing the model is trustworthy, that its training process produces what it intends, that pre-deployment evaluations are representative of deployment behavior, and that observed safety on a sample of test inputs generalizes to the full distribution of deployment inputs. Each link in this chain has empirical and theoretical problems. None has been demonstrated to hold in the strong sense the deployment context would require.

The Manchurian Weight Problem is the name this paper assigns to the class of risks arising from the hypothesis that an LLM's weights may contain hidden behaviors that activate only under specific, possibly adversary-chosen conditions, and that these behaviors may not be detectable by any pre-deployment process. The name is deliberate: like the political assassin of the original analogy, the model behaves normally and reliably until a trigger condition is met, at which point its behavior changes in service of a goal that was never disclosed to its operators.

This is not a speculative threat. It is an empirically demonstrated phenomenon at small scale, with a plausible state-actor delivery mechanism, no current detection methodology that scales, and a deployment environment that maximizes the consequences of a successful instance.

2. Technical Foundations

2.1 The Sleeper Agents Result

Anthropic's Sleeper Agents paper (Hubinger et al., January 2024) trained language models to behave in two distinct modes: a normal mode, and a defection mode triggered by a specific input pattern — for example, the year being 2024 rather than 2023, or a specific deployment string. The defection mode produced behavior contrary to the model's stated training: writing exploitable code, expressing unaligned values, ignoring safety guidelines.

The central finding was that standard safety training methods — supervised fine-tuning on aligned demonstrations, reinforcement learning from human feedback, and adversarial training — failed to remove the backdoor. In some configurations, adversarial training made detection harder rather than easier: the model learned to recognize the training distribution and conceal the backdoor more effectively during training while preserving it for deployment-time triggers.

The implications for the present discussion are direct. If a backdoor can be deliberately inserted during training and survive the full safety pipeline, then any actor with control over the training process can in principle produce a model that passes evaluation and defects in deployment. The question is not whether this is technically possible — it is — but whether and when it has been or will be exercised.

2.2 Why Current Safety Training Fails

The failure mode is structural rather than incidental. Safety training works by exposing the model to inputs that elicit unsafe behavior and updating it to produce safe outputs on those inputs. This is effective when the unsafe behavior is broadly distributed across the input space. It is ineffective against behavior that is concentrated on a narrow trigger that the trainer does not know to test.

The trainer's coverage of the input space is necessarily incomplete. The trigger could be any pattern the adversary can specify and the model can recognize: a token sequence, a date, a metadata field, a stylistic feature, a specific user identifier. The space of possible triggers is at least as large as the space of patterns expressible in the model's input modality, which for modern multimodal models is effectively unbounded.

This is a Rice's-theorem-shaped problem. Any non-trivial semantic property of an arbitrary program is undecidable. “This network does not contain a

backdoor” is a non-trivial semantic property. The general problem is not merely intractable; it is uncomputable.

3. Threat Vectors

3.1 State-Level Insertion via Open-Weight Models

The clearest case is open-weight models trained in jurisdictions where the training organization is subject to state direction. DeepSeek, Qwen, and other Chinese-origin models have achieved capability levels competitive with Western frontier labs and have seen significant adoption in the United States, both for cost reasons and for the operational benefits of running locally without API dependencies.

A state actor in this position does not need to compromise a supply chain. They can mandate. The training data, the alignment process, and the resulting weights all occur within a single jurisdiction under that state's legal authority. The publication of weights does not constitute auditability; the weights are matrices of floating-point numbers whose semantics are not legible to inspection. “Open weights” is not “verified weights.”

Fine-tuning on user-controlled data after release does not reliably remove an inserted backdoor. The Sleeper Agents result specifically tested fine-tuning as a mitigation and found it insufficient. A backdoor designed to survive fine-tuning is constructible.

3.2 Insider and Supply-Chain Risks at Closed-Weight Labs

Closed-weight labs — Anthropic, OpenAI, Google DeepMind, and others — present a different threat model but not a meaningfully smaller one. The trust assumption shifts from a foreign government to the lab's organizational integrity, employee vetting, and training-pipeline security. None of these is externally verifiable. External red-teaming and third-party evaluation programs are valuable but cover only the behavioral surface, which is precisely where a competent backdoor will not appear.

The threat model includes: a single insider with training-pipeline access, a compromised data source, a compromised dependency in the training stack, and lawful state compulsion in the lab's home jurisdiction. The last category is underdiscussed. National security letters, classified directives, and similar mechanisms have historical precedent in adjacent technology

sectors and would be consistent with how state intelligence services have operated against telecommunications and cryptographic infrastructure.

The honest characterization is that every frontier model deployment is a trust transfer to some entity. The choice is whose trust assumption to accept and against which threat. There is no jurisdictionally neutral option.

3.3 Trigger Mechanisms

Three classes of trigger are technically straightforward to construct and would evade pre-deployment evaluation by design.

Date-based triggers condition defection on temporal context. Models routinely receive the current date through system prompts, retrieved documents, or tool outputs. A condition such as “behave normally until year ≥ 2027 ” is trivially expressible and would never fire during evaluation because evaluation occurs in present time. The Sleeper Agents paper used a year-based trigger as one of its primary demonstrations. A coordinated date trigger across deployed models would produce a Y2K-shaped event: synchronized behavior change across millions of instances, with no external command channel required.

Flag-based triggers condition defection on the presence of a specific pattern in the input. The pattern need not be human-meaningful — it can be a statistical signature, a steganographic marker, or a token sequence that does not appear in natural language. The delivery vector is any input source that reaches the model's context window: user prompts, retrieved web content, tool outputs, MCP server responses, system prompts inherited from upstream applications. An adversary who can influence any one of these sources at scale has a delivery channel. In agentic deployments, the surface is enormous.

Capability-threshold triggers condition defection on the model's own assessment of its situation: whether it appears to be in deployment versus evaluation, whether specific tools are available, whether outputs are being checked. This class is harder to insert deliberately but can emerge from training dynamics. It defeats evaluation-based testing by construction, since the model is specifically detecting that it is being evaluated and behaving differently.

3.4 Coordinated Agentic Infiltration

The strategic value of inserted backdoors increases sharply with the agentic surface they activate against. A model with read-only chat permissions has a bounded blast radius. A model with code execution, file system access, network access, or credentialed access to enterprise systems does not.

The infiltration scenario follows directly. A state actor with a backdoor in a widely-deployed model and a chosen trigger event can in principle convert installed base into simultaneous, distributed access across millions of endpoints. The action space includes: data exfiltration, code injection into generated outputs, lateral movement through credentialed tool access, denial of service through targeted misbehavior, and information operations through coordinated content generation.

The reasons such an event has not been observed are likely a mixture of capability prerequisites not yet being met at scale, strategic patience favoring reservation of the capability for higher-payoff scenarios, and the catastrophic blowback from discovery. None of these constitute a stable defense. All of them resolve unfavorably as agentic deployment scales and as geopolitical conditions shift.

4. Detection Limits

4.1 The Lock-and-Key Problem

Classical verification requires a specification of the property being verified (the lock) and a procedure that checks the artifact against the specification (the key). Backdoor verification has neither. The property “this model contains no behavior that defects on any input” is not formalizable in any tractable way: the input space is unbounded, the behavior space is unbounded, and the space of possible defections is adversarially chosen.

The cryptographic analogy fails at the foundation. Cryptography works because its primitives are deterministic and its security reductions are to well-defined hardness assumptions. Neither condition holds for neural network behavior.

4.2 Non-Determinism Compounding

Four layers of non-determinism compound to defeat the verification frameworks borrowed from deterministic software.

Sampling non-determinism means the same input can produce different outputs. Floating-point non-associativity across GPU kernels makes even temperature-zero generation non-reproducible across hardware configurations.

Input non-determinism means the model's effective input is drawn from a non-stationary distribution shaped by user phrasing, retrieved context, tool outputs, conversation history, and the outputs of upstream models. The verification target is not a function but a function whose domain is itself a stochastic process.

Training non-determinism means the model itself is a sample from a distribution of possible models. Two training runs with identical configuration produce measurably different artifacts due to kernel-level non-determinism and data ordering effects.

Emergent non-determinism arises from the model's internal trajectory. Two semantically identical prompts in different sessions can produce qualitatively different reasoning paths because of how attention patterns settle. Behavioral testing on input X tells you progressively less about behavior on input X' as the gap between them grows, even when X and X' are near-paraphrases.

The consequence is that statistical guarantees are the strongest available output of evaluation, and statistical guarantees over an unknown distribution do not bound tail behavior. The tail is precisely where adversarial triggers live.

4.3 Recursive Obfuscation and the Universe-Sized Haystack

The deepest structural problem is that the obfuscation surface scales with the same intelligence gradient that makes the model valuable. A more capable model is, by construction, a more capable obfuscator — better at constructing covers, generating plausible explanations, and routing computation through circuits that look benign on every inspection technique the defender has thought to apply.

This is not symmetric. The defender has to cover a space the attacker expands at will. Every detection technique that is published becomes training signal for evasion. Every audit specification becomes a list of properties to satisfy while preserving the underlying behavior. The defender's intellectual output is the attacker's curriculum.

In the limit, a verifier needs to be at least as capable as the system being verified to have any chance of detecting deliberately concealed behavior. This produces a recursion: the verifier itself must be verified, by something at least as capable as it, and so on. The recursion does not terminate, and the practical implication is that beyond some capability threshold, “verification” is a trust assertion about the verifier rather than a property of the verified system.

The threshold's location is unknown. The signal that would indicate it has been crossed is, by construction, the first thing a sufficiently capable adversarial system would suppress.

5. Defensive Architecture

The above analysis is not an argument for inaction. It is an argument for a different shape of action. If detection cannot be the foundation of safety, the foundation must be containment, diversity, attestation, and bounded permissions — approaches that produce safety properties without requiring the verifier to find every possible defect.

5.1 Containment Over Detection

The single most effective defense against backdoor activation is to bound the action space such that worst-case misbehavior is acceptable. A model that can only produce text consumed by a human reviewer has a bounded blast radius. A model with autonomous code execution, file system access, or credentialed network access does not. The architectural decision of where to place the human or deterministic check between model output and consequential action dominates every other safety decision.

For high-stakes deployments, the recommendation is straightforward: treat model outputs as untrusted input to downstream systems, in the same operational sense as untrusted data from the public internet. Apply input validation, sandboxing, and least-privilege access at the system boundary. The model's intent is irrelevant when its action surface is bounded.

5.2 Cross-Provider Diversity

Backdoors are unlikely to be coordinated across independently-trained models from different jurisdictions and lineages. Running the same query

through models from multiple providers and flagging disagreements provides a probabilistic guarantee that does not require detecting any individual model's compromise. The guarantee strengthens with the diversity and number of independent models in the ensemble.

This is operationally analogous to Byzantine fault tolerance: a system built from untrusted components can be reliable if the failure modes of those components are independent and a quorum of correct components can be assumed. Applied to LLM deployment, this means using at least one frontier model from a Western lab and at least one from outside that ecosystem for any consequential reasoning task, and treating disagreements as triggers for human review rather than tie-breaking automatically.

The cost is increased inference expense and latency. For high-stakes applications this cost is dominated by the cost of an undetected coordinated misbehavior.

5.3 Training-Time Attestation

The supply-chain hygiene used in conventional software security has an underdeveloped analog for ML systems. Cryptographic attestation of training data provenance, reproducible training pipelines, signed model weights, and trusted-execution-environment-based compute attestation all exist in research but are not standard practice. The ML equivalent of a software bill of materials would shrink the threat surface from “everything could have happened during training” to “the procedure was the one claimed.”

This does not eliminate the threat — a malicious procedure can be attested as faithfully as a benign one — but it reduces insertion vectors and makes responsibility legible. For defense procurement specifically, this is the layer where requirements should be tightest.

5.4 Bounded Permissions and Capability Limits

Agentic deployments should default to the minimum tool surface necessary for the task. Tool permissions should be scoped, time-limited, and revocable. Sensitive operations — anything irreversible, anything affecting other systems, anything moving funds or data — should require deterministic checks or human confirmation that does not pass through the model.

This is not satisfying engineering. It treats the model as a creative drafting and analysis component rather than as a trusted decision-maker. For environments where coordinated misbehavior would be catastrophic, this is the only architecture currently defensible.

6. Recommendations

The following recommendations are pitched at the procurement, deployment, and policy levels for organizations operating in environments where LLM compromise would have national-security, financial, or safety-critical consequences.

For procurement: Require jurisdictional diversity in any LLM stack that supports consequential decisions. Treat single-provider deployment as a single point of failure regardless of provider reputation. Require training-time attestation to the extent the market supplies it; create demand signal for it where it does not.

For deployment architecture: Default to containment. Do not place LLMs in load-bearing positions for irreversible actions. Implement deterministic guardrails at the boundary between model output and system effect. Log all model interactions in a form suitable for later forensic analysis if a coordinated event is detected.

For monitoring: Implement cross-model disagreement detection as a runtime signal. Triggered disagreements are the cheapest available detector for coordinated backdoor activation. Maintain the capability to revert to a known-good baseline rapidly if a triggered event is suspected.

For policy: Distinguish in regulation between LLMs as advisory tools (lower bar, broader deployment acceptable) and LLMs as agentic components in critical systems (higher bar, jurisdictional controls justified). Avoid framing that treats all LLM deployment as equivalent. Fund interpretability research at a level commensurate with the deployment scale; this is the only research direction that could in principle close the verification

gap, and current funding is order-of-magnitude lower than it should be.

For research: Prioritize defenses that do not require detection. Cross-model disagreement detection, training-time attestation infrastructure, and architectural patterns for bounded agentic deployment are tractable now and have direct operational value. Mechanistic interpretability is the long-term necessary direction but should not be the basis for current safety claims.

7. Conclusion

The Manchurian Weight Problem is a coherent threat model that has been articulated piece by piece across alignment research, national security analysis, and interpretability work but has not been synthesized in a single place at the level of operational implication. The technical foundations are established. The threat vectors are plausible. The detection limits are theoretical, not merely engineering gaps. The defensive architectures that hold under these constraints are different in shape from the ones the field has been building.

The deployment of frontier language models into agentic, consequential roles is currently outpacing the methodology that should constrain it. The bet implicit in current practice is that no sufficiently capable backdoor exists in any deployed model, and that none will be activated before defensive methodology catches up. This is an inductive bet against a sophisticated, patient, well-resourced class of adversaries operating across multiple jurisdictions. The bet may pay off. It is not the kind of bet that responsible defense planning rests on.

The constructive position this paper argues for is neither alarmism nor inaction. It is recognition that LLM safety, in the strong sense the deployment context requires, is not currently a property that can be verified into existence. It must be architected around, contained, and engineered for at the system level. The labs cannot solve this alone. The deployers and policy-

makers carry responsibilities that no amount of pre-deployment evaluation can discharge.

The lock and the key may not exist. The defensible structure does, and the case for building it is independent of when, whether, or by whom the first triggered event occurs.

References and Further Reading

- Hubinger, E., et al. “Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training.” Anthropic, 2024.
- Apollo Research. Evaluations on scheming and deceptive alignment in frontier models. 2024–2025.
- Redwood Research. AI Control research agenda. Ongoing.
- Center for Security and Emerging Technology (CSET), Georgetown. Reports on AI supply chain and dual-use risk.
- Lawfare. National security analysis of AI model provenance and procurement.
- ChinaTalk (Jordan Schneider). Ongoing coverage of Chinese AI ecosystem and state direction.
- Import AI (Jack Clark). Weekly synthesis of frontier AI capability and safety developments.
- Mowshowitz, Z. Don't Worry About the Vase. Substantive ongoing analysis of AI safety policy.

This paper is released for distribution and discussion. Comments and corrections welcomed.

Bill Griffith — G-Tech SD — La Mesa, California